DAENet: Making Strong Anonymity Scale in a Fully Decentralized Network

Tianxiang Shen*, Jianyu Jiang*, Yunpeng Jiang, Xusheng Chen, Ji Qi, Shixiong Zhao, Fengwei Zhang, Xiapu Luo, and Heming Cui, *Member, IEEE*

Abstract—Decentralized approaches are promising to eliminate centralized vulnerabilities (e.g., single point of failure) of anonymous communication systems. However, existing anonymous systems either provide a limited level of anonymity under global passive attacks (e.g., Tor), or use strong network synchronization to protect users, which exposes a large attack surface for active attackers to halt by intervening packet transmission (e.g., Dissent). We present DAENet, a fully decentralized anonymous communication system that provides both sender-receiver unlinkability under global passive attacks, and sender/receiver anonymity under active attacks. DAENet leverages the confidentiality and integrity protection of Intel SGX to ensure trustworthy message shuffles at each DAENet host. However, as we will show, naive processing of message shuffles within SGX still leaks sender identities under active phishing attacks. To overcome such vulnerability, we retain the use of distributed hash table (DHT) for efficient message routing, but use randomly generated shared secret to formulate different routing circuits in each communication round, preventing active attackers from tracking and revealing sender identities. We show that our system is robust in the case of machine failure and is scalable with moderate latency when running in a cluster of 10,000 participants, making it an attractive new design for decentralized anonymous communication. All of DAENet's code and evaluation results are available on github.com/tdsc0652/dae-net.

1 Introduction

The Internet allows convenient communications between users, but it also incurs great concerns about anonymity since communications can be censored by security agencies and the Internet Service Providers (ISP). These adversaries can learn the identities of users who are talking to each other by recording, tampering, and analyzing communication links [1], [2], [3], [4], [5]. For example, the National Security Agency (NSA) collects the telephone number of phone calls and how long the call lasted to reveal who is talking to whom [6]. Some countries intercept massive amounts of raw Internet traffic and block anonymous services to ease censorship [7].

Formally, three kinds of attacks have been targeting at anonymous services. **(1) Passive attackers** determine if two users are in a conversation by observing network traffic and users' metadata (e.g., packet size and packet rate). The most powerful passive attacker, Global Passive Attacker (GPA) can observe the whole network and record all network packets and their metadata. **(2) Active attackers** can forge, drop or duplicate network packets, and further de-anonymize users by analyzing the changes of network status. Note that a user may also collude with active attackers to gain profit. **(3) Targeted DoS Attackers** completely turn down core servers of an anonymous communication system (e.g., centralized directory server/bridge in Tor) by repeatedly sending dummy messages, or blocking users' accesses to centralized servers by dropping their packets. Therefore, users can only use unsafe communication services censored by powerful adversaries. Conventionally, **(1)** and **(2)** are called traffic analysis attacks. Communication systems that can defend against both attacks are considered to be equipped with strong anonymity.

Existing practical anonymous communication systems can be classified into two categories: relay-based systems and shuffle-based systems. Relay-based systems (e.g., Tor [8] and ShadowWalker [9]) forward messages through a circuit of relays, constructed statically and randomly, to hide the originators of messages. Since relays of the circuit are distributed in the whole network, simply blocking a portion of the relays cannot suspend the whole anonymous service, which defends against targeted DoS attacks. Unfortunately, these systems cannot defend against GPAs because GPAs can learn the originator of a message by collecting output/input packets of all relays, ordering the packets and analyzing all possible routing circuits of a particular message [10].

Second, shuffle-based systems (e.g., Loopix [11], Dissent [12], Karaoke [13], Riposte [14], Miranda [15] and Yodel [16]) enable stronger anonymity guarantee by utilizing centralized servers to collect messages from users and apply cryptographic shuffles to hide the origin of messages. To prevent malicious shuffle servers from invalidating the anonymous service, these systems either use layer-based verifiable shuffles to make sure the majority of the centralized servers are trustworthy [11], [17], [18], [19]; or they apply cryptographically indistinguishable shuffles (e.g., using gamble circuit [20] and holomorphic encryption [21]) to avoid any information leakage of the shuffle process [14], [22]. Moreover, to defend against active attacks (e.g., rate-controlling attacks [23]), these systems typically apply a synchronized approach: each client must send exactly one message at an interval to hide the differences of message metadata (e.g., sending rate). Although these systems achieve strong anonymity guarantees against both global passive attacks and active attacks, they can be blocked by simply dropping some or even one message [13] to the centralized servers.

As far as we know, none of these systems tackles targeted

* *Contributes equally to this work.*

DoS attacks and provides strong anonymity against passive and active attacks at the same time with practicality, and we owe it to two reasons. First, defending against targeted DoS attacks forbids the usage of centralized servers and calls for decentralized approaches, and it is not cost-efficient to enable trustworthy message shuffles distributively. The main reason is that guaranteeing the integrity of shuffles of one server is already costly, therefore applying them to all participants without a set of centralized servers produces prohibitive performance penalty.

Second, since the integrity of distributed participants cannot be guaranteed, compromised participants and global attackers can conduct active attacks by intervening in the packets transmission process, such as controlling message rate or dropping packets [23]. Shuffle-based approaches tackle this problem by enforcing each participant to send exactly one message at an interval with strong network synchronization, but enforcing strong synchronization in a fully decentralized network is non-trivial and cannot achieve in a cost-efficient manner [24]. In fact, the original DC-Net [25] makes use of cryptographic shuffles and all-to-all broadcast, which are computationally expensive and consumes heavy network traffic respectively. As a result, DC-Nets' end-to-end communication latency increases exponentially and they typically support less than hundreds of users in practice.

Recently, Trusted Execution Environment (TEE) such as Intel SGX has been applied in various security domains to guarantee integrity and confidentiality of execution code and data. By running the decentralized communication protocol within SGX enclaves, it is potentially beneficial to provide a stronger anonymity guarantee. For example, SGX-Tor [26] hides the identifiers of hidden services and circuits of Tor by storing the entry relays of the service in SGX enclaves. SGX-Tor is the first practical work that raises the bar for network adversaries by using SGX. However, SGX-Tor is still vulnerable to passive and active attacks and requires centralized directory servers to construct circuits and conduct SGX attestations.

We present DAENet[1], the first practical anonymous messaging system based on SGX that can defend against both traffic analysis attacks and targeted DoS attacks. Building DAENet is not just simply running a distributed network in SGX. First, although SGX guarantees code integrity and data confidentiality, network-level adversaries can still observe network traffic outside enclaves and analyze the originators of messages. Second, active attackers outside the enclave can still drop or delay network packets. Even worse, active attackers might completely abort from its services and withdraw from the network to hurt the liveness of the system.

DAENet defends against passive traffic analysis attack with a *stealthy Peer-to-Peer (P2P) network* that hides the identities of participants when there exists no active attacks. A participant joins the network and connects to a series of random members. The identity of the participant and its neighbors' information are encrypted inside SGX, such that malicious attackers and even the participant's malicious host cannot learn the location (i.e., identity) of that participant in the network. To hide network message patterns, we enforce trustworthy message shuffles and use a dead drop abstraction that randomly chooses one node for exchanging messages between a sender and receiver. With the above-mentioned designs, we prove that our stealthy P2P network produces oblivious packets transmission (§5) for all participants.

To defend against active attacks, instead of using network synchronization, DAENet adopts an efficient detection-based design to detect malicious drop of packets. Each participant periodically sends self-loop packets to a random neighbor and routes back. After that, the participant consults each on-path relay to send back a receipt, proving that it honestly forwarded the loop packet rather than maliciously dropped the packet. This process is executed within SGX enclaves, malicious participants cannot distinguish and intervene in this process. Moreover, since DAENet encapsulates packets in the same format, network administrators cannot distinguish a self-loop packet and subvert the detection protocol.

We implement DAENet with 5.2k LoC in C++ on linux. DAENet handles member admission and helps participants initialize conversations with a membership protocol (§4.1) and secure dialing protocol (§4.5) respectively. DAENet also tolerates network churn and machine failure to guarantee the liveness of its anonymous service.

In conclusion, we make the following contributions:

- This paper analyzes the potential attacks and defenses of existing anonymous messaging systems and discuss the use of Intel SGX for improving security guarantees.
- DAENet is the first practical anonymous network that defends against both traffic analysis and active attacks. We give an end-to-end anonymity evaluation to prove that our approaches are effective.
- DAENet is efficient to scale to a large number of participants. DAENet incurs 2.2s latency with 10,000 participants. Loopix [11], a state-of-art shuffle based system, incurs 6X $\sim$ 7X higher latency than DAENet.
- DAENet is robust to machine failure. After killing 30% participants, DAENet can recover within a short time (less than 5 seconds).

The remaining of this paper is structured as follows. §2 describes the background of low-latency anonymous communication systems and TEE. §3 gives a high-level overview of DAENet, presenting the roles, network topology and security primitives of DAENet. §4 describes detailed DAENet's anonymous protocols. §5 gives an end-to-end security analysis. §6 is the performance evaluation of DAENet. §7 introduces the related work, and §8 concludes our work.

2 Background

2.1 Anonymous Communication Systems

Tor [8] is the most popular anonymous communication system ever deployed on the Internet. Due to its popularity and transparent development processes [30], many researchers have explored attacks that can de-anonymize Tor users and hidden service providers by monitoring the network traffic. Recent attack directions for Tor include website fingerprinting [31], [32], [33], [34], traffic correlation [35], [36], [37], [38],

1. DAENet is for a **D**ecentralized, **A**nonymous and **E**fficient network.

Category		End-to-End Latency (s)	Anti GPA	Anti-Reveal Circuit ID	Anti-Packet drop (liveness) / (Anonymity)	Anti Targeted DoS
Relay-based	Tor [8]	0.25 ∼ 1.5	×	×	√ / ×	×
	SGX-Tor [26]	0.525 ∼ 3.15	×	√	√ / ×	×
	ShadowWalker [9]	> 4	×	×	√ / ×	√
	AP3 [27]	——	×	×	√ / ×	√
Shuffle-based	Loopix [11]	6.8	√	√	√ / ×	×
	Riposte [14]	> 3600	√	√	× / √	×
	Dissent [28]	1.3	√	√	× / ×	×
	Atom [29]	30.0	√	√	√ / √	×
	Karaoke [13]	6.0	√	√	× / √	×
	DAENet	2.2	√	√	√ / √	√

Table 1: Comparison of DAENet to relay-based and shuffle-based systems. "√" indicates that the system can handle such vulnerability, while " × " is on the opposite.

congestion attack [39], [40] and targeted DoS [41], [42], [43]. Meanwhile, researchers also propose methods to enhance Tor's security by optimizing the bandwidth report [44] and monitoring circuit construction [45]. Recently, SGX-Tor [26] explores a new direction to enhance Tor's anonymity by protecting sensitive metadata within trusted hardware (e.g., Intel SGX [46]). Different from Tor-related designs, other relay-based anonymous communication systems, such as ShadowWalker [9] and AP3 [27] are built upon a structured P2P network to eliminate the concern of targeted DoS attacks. However, all these systems are not designed for achieving strong anonymity guarantees against traffic analysis attacks and have poor anonymity guarantee when powerful active attacks occur.

On the other hand, shuffle-based approaches are promising to defend against strong global passive attackers. The most popular shuffle strategy *onion mixnets* is leveraged by numerous anonymous communication systems including Loopix [11], [13], [29], [47]. Typically, an onion mixnet encrypts a sender message multiple times by using the public keys of all on-path mix servers, and shuffles messages at each mix server to mess up the order of input and output messages, such that a network eavesdropper cannot distinguish the real routing circuit of a particular message. Unfortunately, these systems assume several simplifying assumptions: **(1)** a stable network environment where operations can execute in synchronized batches, **(2)** a fixed set of mix servers and a majority of them are benign. It's non-trivial to protect such centralized onion mixnets from DoS attacks while maintaining acceptable end-to-end communication latency. For example, while Riposte [14] handles active attacks, it assumes that clients are willing to tolerate hours of messaging latency. Dissent [28] provides low-latency communication but it reports poor fault-tolerance under active attacks: if a server goes offline, Dissent's protocol completely halts until all servers are available again which severely limits Dissent's practicability.

2.2 Structural P2P Network

DAENet uses Chord [48], a structured peer-to-peer network topology as a foundation for anonymous messaging. In Chord, each participant is assigned with an identifier when it first joins the network. The identifier space is within $[0, 2^b)$, and b is chosen according to the scale of network. Each participant knows only a fraction of other participants in the network. Specifically, for a participant with id idx, she is connected to b neighbor nodes who has the closest identifier to $idx+2^i (0 \leq i < b)$. In DAENet, the neighbor nodes of a participant are called successors and the participant itself is called the predecessor of all her neighbor nodes. Also, not all slots in the identifier space have to be used: each slot in the identifier space, named as KEY_j, is mapped to a numerically closest participant node. (i.e., *Map* $(KEY_j) = ClosestNode_{id}$)

Such topology enables efficient routing in a decentralized network. To send a message, a participant first checks if it is directly connected to the destination of the message and sends it directly if so. Otherwise, it sends the message to her furthest neighbor who is numerically closest to the destination. Chord requires only $\mathcal{O}(\log \mathcal{N})$ steps for a message to reach the destination, which makes it suitable for building distributed network with large amounts of participants.

2.3 Privacy Enhancement through SGX

Intel SGX. Intel Software Guard eXtension (SGX) [49], [50], [51] is a popular security hardware available on commodity CPUs. It provides secure execution by putting data and executing code inside a container called *enclave*. The *enclave* is isolated from privileged software such as the operating system (OS), firmware and hypervisor, so that the protected code/data cannot be easily tampered with. Besides, SGX also provides remote attestation [52] to verify that a particular piece of code is running in a genuine SGX-enabled host.

Usage of SGX in DAENet. We leverage three features provided by Intel SGX to enable stronger privacy guarantee: First, we use SGX to regulate behaviors of participants in the network. SGX provides local/remote attestation service to help admit clean participants and verify the code integrity. By running DAENet's protocol within SGX, we ensure that all participants are running the correct protocol rather than seek to misbehave. Second, we use SGX to simplify our security protocol thus achieving better performance. For example, DAENet ensures correct shuffling of network packets within SGX-enabled hosts. Hence, we do not need to use computational-heavy verifiable shuffle [18] to verify the correctness of the shuffling results. Third, we ensure confidentiality of decrypted plaintext by using SGX, so

that malicious hypervisors or OS cannot see the sensitive information of network packets, such as the circuit ID.

DAENet v.s. SGX-Tor. SGX-Tor proves the feasibility of running SGX-enabled hosts to improve the security model of anonymous communication systems. SGX-Tor improves the original Tor's security guarantee, preventing malicious Tor relays from gaining private information of Tor components, such as circuit identifiers and hidden service identifiers. Although SGX-Tor mitigates many attacks against malicious Tor components, it cannot defend against network-level adversaries, potentially preventing it from being a choice of users who value strong privacy.

DAENet leverages SGX to prevent private information leakage and regulate participants' behaviors, similar to SGX-Tor. Moreover, we further improves SGX-Tor's security model by protecting participants from network-level adversaries, such as GPAs that conduct traffic analysis attack and active attackers that maliciously drop and delay network packets. Although DAENet incurs slightly higher end-to-end communication latency compared to SGX-Tor (shown in Table 1 and evidenced in §6.1), we believe that users may tolerate DAENet's moderate latency to achieve stronger privacy guarantee in anonymous communication.

3 Overview

We begin with describing the threat model and security goals of DAENet, and introducing the roles of participants in the network that facilitate anonymous communication. Then we give the privacy approaches to enable anonymity in a fully decentralized network.

3.1 Threat Model and Security Goals

3.1.1 Threat Model

We consider sophisticated and well-resourced adversaries in the network, who attempt to determine if two participants are communicating, given that the message sender or receiver may collude with the adversaries. Therefore, we consider adversaries with two distinct capabilities: global observation and traffic control. DAENet allows arbitrary number of compromised participants in the network, but requires an adequate number of honest participants to ensure message deliveries. Specifically, DAENet requires at least $\mathcal{O}(\log \mathcal{N})$ out of $\mathcal{N}$ honest participants, in which the coefficient depends on the *rounds* of communications and the total number of participants. Similar to other SGX-enabled systems [26], [53], SGX firmware and the code running in SGX are trusted, SGX-related side-channel attacks (e.g., cache and timing attacks) are out of the scope of this paper.

3.1.2 Security Goals

The main purpose of running anonymous protocol in a fully decentralized network is to remove the targeted DoS concern of centralized services. Moreover, to enable stronger privacy guarantee, we additionally handle the following three critical vulnerabilities under either passive or active attacks.

Protecting against GPAs. Global Passive Attacks (GPAs) are proven to be the strongest passive attack in the network, which can eavesdrop on network traffic exchanged among all the participants, in order to find a message path of a communication. To determine if two participants are in a communication, GPAs may conduct prefix hijacking [54] to intercept network traffic, or off-path statistical analysis [55]. For example, in a typical traffic analysis attack, GPAs inspect the traffic within the network and keep observing the load of each participant. Based on the observation that overall traffic patterns are not particularly distorted in each hop, GPAs correlates the time of incoming and outgoing messages. Meanwhile, GPAs can learn the emitting rate of network packets at each host to distinguish a potential message sender.

Detecting eclipse attack. Eclipse attack is a major vulnerability in a p2p system. With the help of SGX, DAENet prevents adversaries from forging or duplicating network packets, including communicating messages or control messages for maintaining a structural network. However, the adversary might arbitrarily drop network packets among participants to partition a fraction of participants from the network, in order to halt the system liveness. A structural p2p network relies on nonstop sending of control messages to refresh membership and maintain its topology. If some control messages are maliciously dropped, the membership will get partitioned and communications will fail due to incomplete message deliveries. In a partitioned network, the anonymity set is smaller and the adversary will have a larger chance to de-anonymize targeted participants (§4.3).

Preventing phishing attack. As a fully asynchronous messaging system, DAENet may become the target of network adversaries who collaborate with participants and control the message emitting rate to reveal the identities of another communicator in the same conversation. In this attack, a compromised message receiver keeps holding connection and communicating with another participant in the network, while the network adversaries delay/drop packets between the sender and receiver to reveal the routing circuit, based on the observation of whether the compromised receiver has received the packet from the sender in time or not (§4.4).

Specifically, in this *phishing attack*, a malicious receiver R collaborates with active attackers to reveal the identity of sender S. Denote circuit $||C_i|| \Leftarrow < S, P^1, P^2, ..., P^{i-1}, P^i, R >$ as the routing circuit that links the malicious receiver R and victim sender S. Since the network structure is explicit to adversaries with global view (§5.4), R can periodically, yet slowly drops messages from her predecessors. If R drops an application message from one of her predecessor and receives no message from S in this communication round, then R can determine that this predecessor is actually P^i - the participant that acts as the previous hop in $||C_i||$ that links R and S. By repeating this process, the malicious receiver R will ultimately reveal the identity of sender S when blocking all the packets from S's predecessor does *not* effect receiving the message from S.

Note that the phishing attack applies to compromised senders as well, who collaborates with active attackers to reveal the identity of a targeted receiver.

3.2 Participants as protocol parties

In DAENet, all participants equally act as protocol nodes, with no designated parties, such as administrative servers

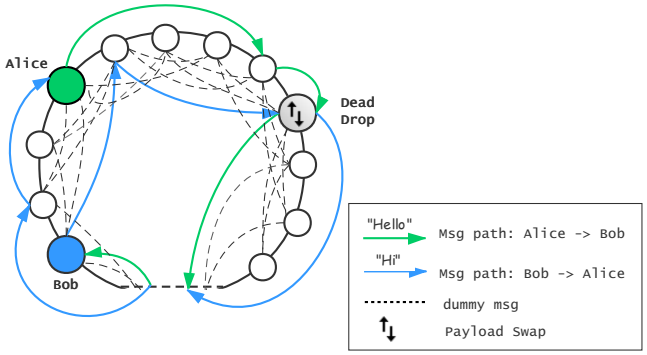


Figure 1: Example of DAENet dead drop messaging. In each round, Alice and Bob provider separately sends two close-loop messages while exchanging the payload at a designated dead drop node.

or message boxes in previous anonymous communication systems [11], [28]. A participant can play multiple roles at a given time, determined by whether it is communicating or not. Therefore, targeted DoS attackers cannot distinguish a specific participant who is playing a specific role. There are totally three roles in DAENet: relay, session node (i.e., sender/receiver) and dead drop node.

Relays are idle participants. They are not in any conversations and only responsible for forwarding packets in the network, including both application packets (e.g., Alice's instant messages) and underlying p2p control packets (i.e., messages for maintaining the structural topology).

Session nodes are in private conversations and each pair of session nodes shares a secret used for communication. To communicate with each other, session nodes use the same Pseudo Random Number Generator (PRNG [56]) with their shared secret to generate a pseudorandom sequence of keys, and these keys are mapped to a deterministic set of participants in the network. DAENet proposes a secure dialing protocol to establish a secure communication channel and negotiate communication details (e.g., the shared secret) between two session nodes (§4.5). A participant acts as either a relay or a session node in DAENet.

Dead drop nodes exchange packet content between any pair of session nodes. To initialize a set of dead drop nodes, two DAENet participants first negotiate a randomly generated shared secret. The shared secret is used for generating a sequence of $DeadDrop_keys$. Since DAENet enables deterministic *KEY-ID* mapping (§2.2), $DeadDrop_keys$ are deterministically mapped to a series of nodes. Hence two session nodes agree on the same sequence of dead drop nodes in the network. All participants can act as dead drop nodes in DAENet. Also, the duty of a dead drop node is ephemeral and will become invalid as soon as the dead drop completes packet exchanging. (§4.4)

Figure 1 shows a flow of dead drop messaging in a communication round. Alice and Bob both send their messages to a designated participant (i.e., dead drop node) in the network, with reference to the shared secret. The dead drop node waits for two messages coming and then exchanging the packet content and sending them back to originators.

3.3 Privacy Approaches

Message shuffle. DAENet's topology is explicit to GPAs. Adversaries can keep observing global traffic flow and deducing all the neighbors (i.e., successors/predecessors numerically) of a node through statistical analysis (§3.1.2). Nevertheless, DAENet ensures that, even with an observed topology, GPAs still cannot correlate any pairs of sender/receiver in conversations because of the trustworthy message shuffles at each SGX-enabled host.

Denote $\mathcal{N}$ as the total number of participants, the membership protocol ensures that Alice's location (i.e., IP address) is known to only $\log \mathcal{N}$ participants that precede her (i.e., predecessors). Upon receiving a network packet sent by one of Alice's predecessors, Alice first decrypts and parses the packet. If the packet is a dummy message, then Alice directly discards that packet; Otherwise, if Alice is not the destination node, it searches for the next hop by finding a numerically closest successor node in her routing table. Alice maintains a shuffle pool for each successor node (i.e., neighbor) in her routing table. If the searched next hop is the ith neighbor of Alice, then the packet is pushed to the ith shuffle pool belongs to her ith successor. In each protocol run, Alice pulls packets from each successor's *shuffle pool* with a probability. If Alice does not pull a packet from the ith neighbor's shuffle pool, then it will send a dummy message to the ith neighbor. Note that Alice may hold none packets in her ith *shuffle pool*, if so, Alice will directly encapsulate and sends a dummy message to her ith neighbor.

Round based Dead drop messaging. Similar to other DHT based lookup schemes, DAENet's routing is deterministic which means that the routing path from Alice to Bob is fixed. This poses a privacy threat to sender anonymity: an active attacker might collaborate with the message receiver and reveal sender identity by reconstructing the routing path between them. In the active attack, a malicious receiver holds a connection with a sender, and hierarchically drop packets from the last hop to the first hop of the circuit (described in §3.1.2) to reveal the identity of the target sender.

A straw man approach is to detect such attack. However, a detection-based approach is hard to conduct because (1) it's hard to verify a legal/compromised receiver and (2) the active attack is recordable. Even the conversation between sender and receiver is set to expire within a fixed time, when the sender connects to the receiver again, the adversary can start from previous derivation and continue reconstructing the circuit.

To defend against such vulnerability, DAENet uses a sequence of randomly generated *dead drop nodes* to communicate: Dead drop nodes are virtual locations where senders and receivers deposit their messages (original packets), swap message payload and fetch messages (swapped packets) back. To initialize a conversation, two participants first negotiate a randomly generated *shared secret* (§4.5). The *shared secret* is used for generating a sequence of *DeadDrop_keys*. Since DAENet enables deterministic *KEY-ID map* by building on top of Chord, thus the two participants are agreed on the same set of dead drop nodes.

Now the two participants can communicate with each other through these dead drop nodes. Communication

happens in *rounds*. In round i, two participants independently send their message to dead drop node N_i with $DeadDrop_key_i$. Each message is labeled with *session-round* pair in case of wrong payload exchanging. When N_i receives a message m, it stores it and waits for the coming of m_1 which has the same label as m. Then N_i swaps the payload of these two messages and sends them back to corresponding message originators. (§4.4)

4 DESIGN

This section gives discussion on DAENet's design, starting from the membership protocol, and presenting the shuffling strategy to defend against GPAs under consistent membership (normal case). Then we discuss our approaches to preserve anonymity under active attacks (special case).

4.1 Membership Protocol

To use the anonymous service provided by DAENet, users first join the network as participants. When node *i* wants to join DAENet, it first finds a member node through an off-band peer discovery service. We call that member node the *joiner node*. A *joiner node* serves as an attestation server to verify the code integrity. If node *i* passes the attestation, then the *joiner node* will automatically generate an identifier, which indicates node *i*'s location in the DAENet topology.

Specifically, node *i* joins DAENet with three steps. First, *i* creates its DAENet enclave, generates its symmetric key sk_i in the enclave and seals sk_i to local storage. Second, *i* sends a *join* request to the *joiner node*. The *joiner node* does a standard SGX remote attestation and succeeds with a signed report from Intel IAS. Third, the *joiner node* verifies the report, generates an identity of node i and encrypts it with sk_i, and sends both the sealed identifier and attestation report to node *i*. As *i* passes the attestation, it sends a lookup request with its symmetric key sk_i to the *joiner node* to construct its routing table. The *joiner node* helps node *i* constructs its routing table and also notifies a fraction of nodes that precede *i*. Note that sk_i is distributed to all the predecessors of *i*, which is used for encrypting messages that are sent to i.

Utilizing the above approach to admit regulated participants may have a potential risk: the channel to join the network is public to adversaries, thus a node may discover a fake DAENet participant and join a fake DAENet. Also, if a malicious participant is chosen as a *joiner node* and serves as an attestation server to admit new nodes, it might refuse to admit benign nodes or try to admit specific participants (most likely be malicious). DAENet uses mutual attestation to detect malicious *joiner nodes*. A newly joined node will also serve as an attestation server to verify the integrity of its *joiner node*. The mutual attestation is triggered when a *joiner node* sends the attestation report to node *i*, at the same time it provisions a self-attestation request to node *i*. Now node i acts as an attestation server, sends the report of the *joiner node* to Intel IAS and waits for a signed report.

Note that the attestation to a *joiner node* is hardcoded into the membership protocol and the execution is enforced unless the *joiner node* withdraws from the network. Because a malicious *joiner node* will fail to pass the attestation, it helps the new participant take action quickly:

(1) Alert users in the off-band peer discovery service, reduce the confidence of that malicious *joiner node*, or end up contacting with that *joiner node*.

(2) Retry the admission process by switching to a new *joiner node* (hopefully, one that is not malicious).

This policy limits the influence a malicious *joiner node* can do during admission, allowing DAENet to admit trustworthy participants running correct protocol. Note that DAENet can only admit SGX-enabled hosts as participants and will reject hosts without SGX.

4.2 Shuffling: Prevent Circuit Correlation

To ensure the unlinkability between senders and receivers, DAENet's shuffling strategy is designed so that, for any message that traverses a participant, GPAs cannot link another message that precedes or succeeds it and further reconstruct an entire routing circuit of a targeted conversation. We define *Sender-Receiver Unlinkability* as the inability for a GPA to distinguish whether $\{S_{real} \rightarrow R_{real}\}$ or $\{S_{real} \rightarrow R_{other}, S_{other} \rightarrow R_{real}\}$ for real message sender/receiver S_{real}, R_{real}, and other participants S_{other}, R_{other} of GPA's choice.

If a participant of DAENet sends a message immediately after receiving that message from one of her predecessors, this would allow GPAs to link these two messages and further deduce the circuit. DAENet's shuffling protocol requires each participant to maintain *shuffle pools* for each of her successors. For each input message, Alice first decrypts the message by using her symmetric key, recalling that for each message delivery, the sender will encrypt the message by using next hop's symmetric key. Then Alice searches for the next hop by using the identifier of the destination node. If the next hop for that message is the *i*th successor of Alice, then the message is pushed to the *shuffle pool* belonging to the *i*th successor.

In each protocol run, Alice pulls $\log \mathcal{N}$ messages from each of her *shuffle pool* by α, which is the *expected shuffle rate* - a parameter that indicates the probability of choosing a message from a *shuffle pool*. In other words, the *expected shuffle rate* implies whether Alice will send an message to her *i*th successor or not. If Alice does not pull a message from the *shuffle pool* of the *i*th successor, then Alice encapsulates a dummy message and will send that dummy message to the *i*th successor. Note that it's likely for Alice to hold none messages in the *i*th *shuffle pool*. In that case, Alice will directly send a dummy message to her *i*th successor.

More precisely, when Alice receives a message x, she

(1) decrypts x by using her own symmetric key key_A,

(2) discards x if x is a dummy message. Otherwise, runs the Chord lookup protocol to search for the next hop of message x. Let x_id be identifier of the next hop, Alice resets x's packet header to x_id, and push message x to the shuffle pool of x_id (i.e., $pool_id$).

(3) randomly pulls l messages from each successor's shuffle pool with equal probability α.

(4) encapsulates dummy message dmy_i if Alice fails to pick one message from shuffle pool p_i.

(5) encrypts l messages with the symmetric key of corresponding successors and sends them out.

Theorem 1. *Denote $\mathcal{N}$ as the total number of participants and k as the number of empty shuffle pools, Alice pulls messages from*

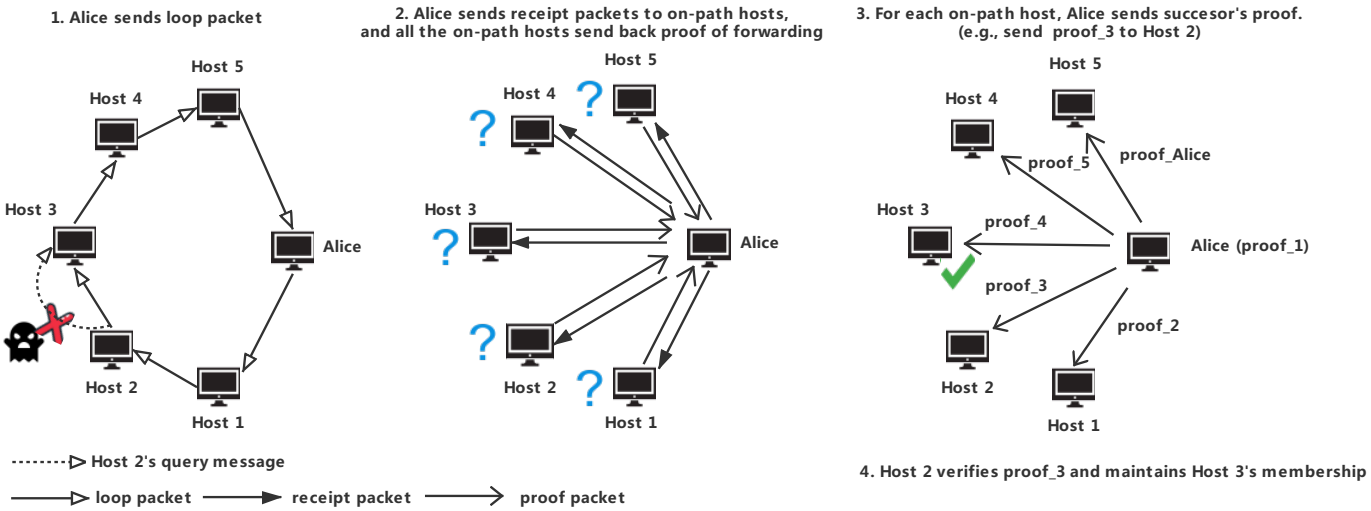


Figure 2: Each participant periodically, and randomly sends loop packets to detect malicious packet drop. Here, even if Host 2's query message to Host 3 is maliciously dropped, after verifying Host 3's *proof of forwarding*, Host 2 determines that Host 3 is actually online, hence maintains Host 3's membership.

$\log \mathcal{N} - k$ *shuffle pools in each protocol run. The pulling process takes the form of Binomial distribution* $\mathcal{X} \sim B(\log \mathcal{N}, \alpha)$ *The discrete probability* α *is the expected shuffle rate. In each round, the expected number of messages and dummy messages for Alice to send are* $\alpha(\log \mathcal{N} - k)$ *and* $k + (1-\alpha)(\log \mathcal{N} - k)$*, respectively.*

We consider the case where GPAs keep observing network traffic and are capable to learn the exact number of messages in Alice's host. We define O_{x,x_1} as an adversary observing Alice's host in which message x arrives, mixes together with in Alice's shuffle pools. The adversary then observes $\log \mathcal{N}$ messages sending out and tries to correlate x with one of the outgoing message x_1, which is from the same circuit of a conversation.

Supposing the adversary has a high confidence of message x being a real message (rather than a dummy message) and wants to correlate x with another real message x_1 from one circuit. The following lemma provides an upper bound on the probability that an adversary correctly link the previously observed message x with one of the outgoing message x_1.

Theorem 2. *Let* y *be the number of messages in a host in scenario* O_{x,x_1}*. Denote the number of non empty shuffle pools in a node as* t*, and let* k *be the number of empty shuffle pools. After shuffling, the probability of correctly linking* x *to one of the outgoing message* x_1 *is*

$$Pr\,(x = x_1) = \frac{\alpha[\sum_{c=1}^{y-t-1} \frac{1}{c} Pr(C_x{=}c)]}{t+k} \tag{1}$$

in which

$$Pr\,(C_x = c) = \binom{\text{y-t}}{c} (\frac{1}{t})^c (\frac{t-1}{t})^{y-t-c} \tag{2}$$

Note that $t + k$ is the total number of outgoing messages from Alice's host. All of the outgoing messages have equal opportunity of being the previously arrived message x, independent of the arrival time of x. This ensures that the arrival and departure time of the messages cannot be linked, so that adversaries learn no sensitive information by conducting traffic analysis. Note that the probability $\frac{1}{y}$ is the upper bound for an adversary to correctly link the input message x and the corresponding output message x_1. Thus, continuous observation of Alice's traffic leaks no sensitive information other than the present number of messages in Alice's host.

We use the above theorems and a security metric *likelihood* to give an end-to-end anonymity evaluation in §5. To conclude, by randomly picking real messages from shuffle pools and disguising unpicked real messages with dummy messages, we obfuscate the adversary's view and decrease the probability of successfully correlating the input/output message.

4.3 Handling Missing Shuffle Messages

Running the shuffling protocol within SGX guarantees the integrity of shuffling operation. However, active attackers might attempt to circumvent the protocol by refusing to cooperate in message sending, result in some loss of shuffle messages. When some shuffle messages are missing, adversaries can conduct eclipse attack to ruin the network construction by prevent participants from obtaining a consistent view of membership.

Although we encapsulate packet within SGX and adversaries (including the malicious host itself) cannot determine whether a sent out message is a dummy message or a real message, they can arbitrarily drop messages in the network to ruin the process of network construction, harass the membership and ultimately deny the service of DAENet.

There are totally two cases: first, if an adversary drops a dummy message or application message, the construction of the network will not be affected. The underlying p2p construction relies only on control messages, dummy messages and application messages are not involved in this process. Second, if some control messages are maliciously dropped by active attackers, the membership will get partitioned if DAENet naively uses one query message to determine whether a participant's neighbor is alive or not because

a participant will remove the "offline" neighbor from its membership list even if this neighbor is actually online.

To prevent such vulnerability, DAENet's participants periodically send *loop packets*. The sender S specifies a random bit b_s and constructs a routing path by selecting the $b_s th$ neighbor as the next destination in each hop, until encountering the sender S itself. Participants at each hop record both b_s and round number as a receipt of forwarding the loop packet. The algorithm of sending a loop packet is shown in algorithm 1.

Algorithm 1: Send_loop(PACKET)

```
S_id ← the identity of sender S
b_s ← random bit generated by S
rnd ← the round of PACKET chosen by S
record(t) ← mark down t as a receipt of forwarding
  PACKET
send(p, dest) ← send packet p to next hop dest
if S_id ∈ this.getNeighborList() then
    send(PACKET, S_id);
else
    record(< b_s, rnd >);
    NextHop = this.getNeighbor(b_s);
    send(PACKET, NextHop);
```

If the loop packet successfully completes its loop back, this indicates that all the participants on the routing path are alive, then the sender will send *proof of forwarding* to all the on-path participants to notify that their successors on this routing path is valid. On the contrary, if the loop packet fails to complete the loop back, this reveals that at least one of the participants on the path misbehaved. Then the sender sends receipt packets to all the on-path participants to ask for a proof of forwarding. Notice that the random bit in the loop packet is originated within SGX, even a malicious host cannot see the plaintext of that bit. Hence the random bit in loop packet is unforgeable. When receives the receipt message from the sender, each on-path participant sends back proof of forwarding independently. The sender verifies the bits in each proofs and reports the verification result back to the participants.

In Figure 2, Host 2's query message to Host 3 is dropped by an adversary. Instead of removing Host 3 from her neighbor list immediately, Host 2 waits for a proof of forwarding within a fixed time interval. When Host 2 receives such proof from Alice that Host 3 forwarded her loop packet successfully, Host 2 learns that her query message may be dropped due to either unstable network environment or malicious active attackers, and will keep Host 3's membership maintained. If Host 3's proof is invalid, then Host 2 can correctly remove Host 3 from her neighbor list and re-fetch a new list. All participants that contain Host 3 will soon be notified within several protocol runs (by running Chord's topology maintenance protocol).

Note that all loop packets in DAENet are stealthy. Each participant periodically, yet randomly send loop packets to check the liveness of on-path nodes, so that adversaries cannot distinguish between a loop packet and a "regular" packet, and drop specific loop packet to prevent sending proofs.

A special case here is when an aggressive active attacker entirely blocks the traffic from a participant to one of her successors. In that case, the loop packet will fail to complete its loop back, and the participant may consider the successor to be offline. Confronted with this attack, we argue that if a participant's query message cannot be replied for a long time, she is more likely to be monitored. To continue her conversation safely, she may rejoin the network to fetch a new list of successors.

4.4 Anti-phishing: Keep Sender Identity Confidential

Straw man approach. As discussed in §3.1.2, active attackers might collaborate with compromised participants to reveal the identity of a targeted sender or receiver by dropping/delaying the traffic. A straw man approach is to detect malicious phishing behaviors in the network. However, detecting phishing attacks in the network is difficult. First, naively setting a threshold ϵ as *time-out* at sender is impractical because we cannot determine an average latency of communication in the network as the number of participants grows and network environment differs in places. If ϵ is too large, the detection threshold is useless; Otherwise, if ϵ is too small, communication becomes hard to carry on in the network. Second, *loop packet detection* does *not* work in this scenario. loop packet is used to prove to a participant that a potentially withdrawn neighbor is online. However, since the sender S does not know the exact or even relative position of malicious receiver R in the network, loop packets cannot tell whether the application message drop is due to an offline receiver or malicious phishing behavior.

To solve this problem, we utilize a *round-based dead drop* design to prevent malicious receivers from revealing the identity of senders. The basic idea of this design is randomly selecting a sequence of participants in DAENet as destinations for corresponding sender and receiver to exchange information in different *rounds*, and enabling full asynchrony to hide messaging patterns. Next, we discuss our *round-based dead drop* design and how we use SGX to hide the access pattern of dead drop nodes.

Round-based Dead Drop Messaging. DAENet enforces communication through a sequence of *dead drops*: Dead drops are virtual locations on DAENet hosts where senders and receivers deposit their messages (*original packets*), swaps message payload from the same conversation and fetch messages (swapped packets) back. To initialize a conversation, two DAENet participants first negotiate a randomly generated *shared secret*. The *shared secret* is used for generating a sequence of *DeadDrop_keys*. Since DAENet enables deterministic *KEY-ID map* by building on top of Chord, two participants can agree on the same set of dead drop nodes.

Now the two participants can communicate with each other through these dead drop nodes. Communication happens in *rounds*. In round i, two participants independently send their message to dead drop node N_i with $DeadDrop_key_i$. Each message is labeled with *session-round* pair to indicate its identity. When N_i receives a message m, she stores it and waits for the coming of m_1 which has the same label as m. Then N_i swaps the payload of these two messages and sends them back to corresponding message originators. The round-based dead drop messaging

is effective to defend against active attacks because (1) communication circuits keep changes by setting different dead drop nodes as destination and (2) decouples the "send-receive" relationship between two participants, thus attacks to reveal participants' identity by monitoring the messaging process (including packet drop) cannot work.

To subvert the dead drop messaging design, a compromised sender might delay her message in a communication round to let the designated dead drop node in that round delay sending back the exchanged message to the receiver. Such misbehavior will not pose an anonymity concern because the adversary cannot distinguish a delayed message from the dead drop node to the receiver with trustworthy shuffles, so that the delayed message cannot be tracked to reveal the identity of the receiver.

At the highest level, communicating through dead drops is not the first design proposed by DAENet. Prior mix-net anonymous networks (e.g., Karaoke, Vuvuzela) leverage similar idea to prevent malicious receivers from directly talking to senders and reveal senders' identities. However, the dead drop design is vulnerable, as the dead drop nodes are potential to be compromised or controlled by active attackers, and the adversaries might also observe the access pattern of dead drop nodes to determine who are in the same conversation.

To achieve privacy even with malicious dead drop nodes, DAENet addresses by three policies listed as follows.
Trusted swapping. All dead drop behaviors are executed within SGX. Since SGX guarantees the confidentiality of decrypted messages in memory, a malicious dead drop node cannot determine whether two messages belong to the same conversation and which communication pairs are being swapped.
Ephemeral duty. Dead drop nodes in DAENet are ephemeral which means they do not need to persist over time. As DAENet works in asynchronous rounds, a dead drop node (agreed on by two participants) is only responsible for handling message swap in current communication round, unless being chosen by the two participants again. Hence, a malicious dead drop node will not always hold the conversation and have no chance to reveal the link between the two participants §5.
Unified output rate. According to DAENet's shuffling protocol, it's difficult for a malicious dead drop node to distinguish whether a received message is a real message or dummy message. Also, DAENet's shuffling enforces unified output rate among honest participants to hide the access pattern of dead drop nodes. Honest participants periodically send a message (either real or dummy) to all her neighbors with a fixed output rate. Hence, if a malicious dead drop node delays or speeds up its emitting rate, it will *not* observe an expected traffic fall or spike to reveal the identity of communicating participants.

4.5 Secure Dialing: Conversation Initialization

A dialing protocol helps two participants (namely, a client and a service provider) initiate their conversation in the network. However, a client needs to start her conversation without being identified, and a service provider needs a secure way to help clients reach her while preserving anonymity. This is handled by DAENet's dialing protocol, shown in Figure 3.

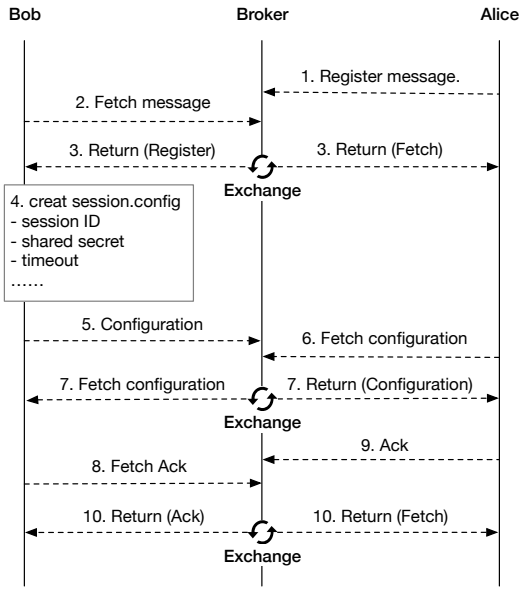


Figure 3: Two participants of DAENet initiates their conversation through a secure dialing protocol.

For the client/service provider, the whole dialing procedure proceeds within three steps through a special dead drop node, namely a *broker*. A *broker* is the service provider's designated virtual location that is responsible for receiving initialization requests in the network. A client who wants to start a conversation with the service provider dials through that *broker*, and negotiates necessary conversation configurations (e.g., the shared secret, session ID, expiry time).

With the help of *broker*, the client and service provider establish a secure channel to start their conversation. Note that in DAENet, the dialing protocol and conversation protocol execute independently and asynchronously. Since the packet size for dialing is totally the same as conversation (limited in 1KB), and the packet format is totally the same, the combination of dialing traffic and conversation traffic could improve DAENet's privacy. Next, we introduce DAENet's dialing protocol from the angel of both the client and the service provider.
For client: To register a service of service provider, Alice (client) first finds the key of Bob from an out-of-bound channel. It could be a database where all DAENet service providers (including Bob) put their *service_key* and a *broker_key*. With the destination being the *broker_key* of Bob (KEY_Bob), Alice encapsulates a *welcome* message containing Bob's *service_key* and sends that message by calling DAENet's *connect()* function. As a structured p2p network, with parameter KEY_Bob, Alice's *welcome* message will finally reach a participant, which is called a *broker* in DAENet. The *broker* verifies the *service_key* to ensure that Alice wants to connect to Bob. In next round, Alice sends a fetch request to the broker to ask for the configuration file for transmission. Within the configuration file, there are totally four elements: $sessionID$, $shared_secret$, $transmission\ rate$ and $time_out$. Last but not least, Alice sends an ACK to the broker to invoice the arrival of the configuration file, and

implies that he agrees with the $shared_secret$ for secrete communication. By this step, the dialing process for a client is completed successfully.

For service provider: As a service provider, Bob makes its service public by releasing its service key (i.e., Bob's public key $PubKey_{Bob}$) through an out-of-bound channel while keeping the corresponding private key $PrvKey_{Bob}$. Then, Bob registers his service to the *broker* with key $hash(PubKey_{Bob})$, using a registration message $< PubKey_{Bob}, identifier, random, Sig >$, where $Sig = Sign(identifier||random, PrvKey_{Bob})$. The *broker* first verifies the signature of this message with the public key. This verification is necessary because a user may maliciously claim that one service key belongs to him and provides the key to DAENet in order to block service of the real owner or get messages intended for the real owner. If the verification passes, the *broker* stores a mapping $< PubKey_{Bob}, identifier >$ in its enclave. When the *broker* receives a *welcome* message from any client in the network, it tries to find out the identifier of the designated service key. If it succeeds, the *broker* notifies Bob for negotiating further communication details.

Through DAENet dialing, Alice now has registered itself to Bob without knowing Bob's location, Bob is aware of a service request from somewhere in the network. With a negotiated configuration file for transmission, Alice and Bob can then carry out communication.

5 Security Analysis

5.1 Analysis of Traffic Analysis Attack

This subsection gives an end-to-end anonymity evaluation to analyze the impact of passive traffic analysis attack in DAENet. As the strongest traffic analyzer, GPAs monitor global traffic and observe messages entering and exiting a participant, in order to link corresponding message sender and receiver.

Thus, we analyze the unlinkability between senders and receivers by using an empirical analysis tool, used by Loopix, to study the correlation probability of two messages in the network. The security metric that we use is called *likelihood difference*, which reveals the probability of linking a leaving message to a sender S_0 in comparison to another sender S_1. Denote the *likelihood difference* as ϵ, the two probabilities that a message is sent by S_0 and S_1 as $p_0 = Pr[S_0]$ and $p_1 = Pr[S_1]$. Our evaluated *likelihood difference* is

$$\epsilon = | \, log(\, p_0 \, / \, p_1 \,) \, | \tag{3}$$

in which p_0 and p_1 can be calculated from Equation (1) and Equation (2).

To study the probabilities, we run DAENet in a local cluster, ranging from 1,024 participants to 8,192 participants that generate and send messages simultaneously with unified messaging rate 50ms. Among the participants, 10% participants hold on communications while the left 90% participants do not communicate. We challenge the two senders S_0 and S_1 to analyze the probability: First, all participants wait for a membership warm-up time until the network becomes steady to test. All the 10% communication holders, except for S_0 and S_1, simultaneously send messages to the network. Then, let S_0 and S_1 encapsulate two messages, tag the two messages and send them to the network as well.

Now that there are two messages sent by S_0 and S_1 in the network which are manually labeled, while the remaining messages sent by other participants are not labeled. At each hop, we track the probability that an exiting message is labeled S_0 or S_1, and calculate the probability of being one of the senders through Theorem 2 (§4.2). As we pick S_0 and S_1 in their final destination, we calculate ϵ in Equation (3).

Varying the parameter of message emitting rate and shuffle rate, we average the evaluation results over 1000 repetitions and illustrate them in Figure 4 and Figure 5. Our experiment shows that the expected *likelihood difference* is small (lower than 0.31).

More participants, stronger anonymity. As we can see from Figure 4, ϵ degrades almost linearly with more participants. This indicates that, by increasing the number of users of DAENet, the anonymity of participants can be further improved. When DAENet scales out to a large number of users, participants in the network process more messages. As all the messages fully mixed in shuffle pools, the likelihood difference of two senders decreases, indicating that GPAs have less probability to link message senders and receivers.

Parameter selection. Figure 5 shows that the expected *likelihood difference* decreases (0.30199 to 0.2409) with decreasing shuffle rate. This figure illustrates that (1) decreasing the probability of pulling a message from shuffle pools (by decreasing the shuffle rate) with respect to the message emitting rate increases anonymity and (2) the shuffle rate has small impact on the anonymity of participants. As the shuffler rate decreases, DAENet requires participants to send more dummy messages. To save the bandwidth cost, we consider shuffle rate = 0.8 to be a good choice in terms of anonymity.

Unlinkability under corruption. We assume all participants to be honest in this experiment. A fraction of compromised participants who try to delay the packets in the network have slight influence on the anonymity. Since we encapsulate all packets in the same format within trusted executing environment, a compromised participant cannot distinguish targeted messages (i.e., labeled message) and delay them to increase the *likelihood difference*.

5.2 Analysis of Active Attacks

In this subsection, we analyze the impact of active attacks in the presence of DAENet. First, we analyze DAENet against targeted DoS attacks when network attackers compromise a fraction of nodes to increase the chances of participants choosing fully malicious messaging path. We continue by evaluating the round-based dead drop design and the security of loop messages.

5.2.1 Resisting targeted DoS Attacks

DAENet can defend against targeted DoS attacks with the assumption that the packets will not be relayed via a fully malicious routing circuit, which is entirely controlled by the adversary. If a routing circuit is fully controlled, the adversary can trivially track all traffic and deduce that the sender and receiver are within a small anonymity set. In

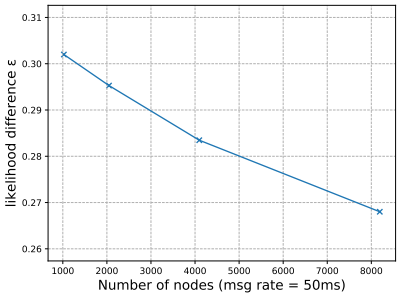


Figure 4: Likelihood difference ϵ depending on the number of participants in the network.

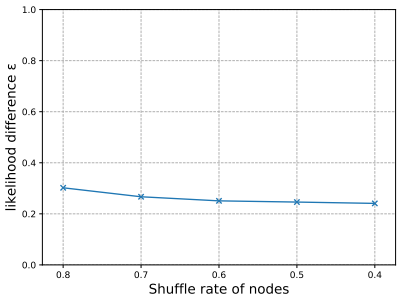


Figure 5: Likelihood difference ϵ depending on the shuffle rate for each participant in the network.

other words, the sender will be one of the predecessors of the entry node of the circuit, and the receiver is considered to be one of the successors of the exit node of the circuit.

Because routing circuits are chosen by the underlying p2p lookup protocol, which is correctly executed within SGX, the only way the adversary can succeed in conducting targeted DoS attack is by adding more compromised nodes. However, the adversary cannot disconnect any honest nodes from being picked in a circuit because the loop messages periodically check the liveness of honest nodes to prevent them from being withdrawn, therefore, the adversary can only increase the probability of being chosen as a relay in circuit.

Denote M_{adv} as the set of compromised nodes controlled by the adversary, N as the total number of nodes in the network and p_m as the proportion of compromised nodes. During the circuit generation process, the probability of choosing a fully malicious routing circuit is

$$Pr(circuit \in M_{adv}) \leq (p_m)^{LogN} \quad (4)$$

Equation (4) indicates that adding more compromised nodes only slightly increases the probability of choosing a fully malicious routing circuit. When the network scales to 10,000 participants, even with a large compromised rate (p_{m1} = 0.8, p_{m2} = 0.5), the probability of successfully conducting targeted DoS is less than 0.05 and 0.0001, respectively. Even with a fully controlled routing circuit, the adversary still cannot distinguish whether a participant is talking to someone else or not. To further de-anonymize a packet sender and receiver, the adversary has to sabotage the entry and exit traffic and collaborate with one party to ensure a conversation indeed happens through this fully compromised circuit. Note that the larger the scale of DAENet, the more expensive the targeted DoS attack, and the lower the probability of success.

5.2.2 Resisting Aggresive Active Attacks

As discussed in Section §4, active attackers can halt the anonymity through dropping and delaying packets, or even collaborate with DAENet participants to launch stronger attacks in the network. Active attackers might try other approaches to subvert the anonymity of participants. We first discuss the security of our approaches against most common and critical active attacks, and then discuss other potential attacks that try to de-anonymize DAENet participants.

Security of Loop Packets. A compromised participant might refuse to receive or forward predecessors' packets. If the dropped packets are application messages, such attack is imprudent because participants can simply wait for a short time and send that message in other communication round, without leaking anonymity; Otherwise, if control messages are dropped, such misbehaves will be detected by loop packets. The preceding participant can force the succeeding participant to formulate a *proof of forwarding* with a receipt to prove itself forwarded her packets.

Loop packets are secure against counterfeiting. Since the receipt must contain a secret key, which is generated by the the loop packet sender and kept secret within SGX, compromised participants and network adversaries cannot duplicate that key to counterfeit the forwarding behavior. Also, the adversary is incapable to determine a loop packet. Because all the loop packets are encapsulated in the same way as genuine messages, and processed through the shuffling. Thus, adversaries cannot seek to predict a loop packet by observing the timing pattern. As the output rate of each participant is unified, adversaries cannot determine loop packets by observing the rate of sent messages.

Defeating DAENet Anti-phishing Protocol. DAENet uses round-based dead drop design to protect senders/receivers from being observed by active attackers who collaborate with compromised participants. As we discussed in §3.1.2, a fixed circuit in a p2p network gives chances to attackers to hierarchically reconstruct the message path. By using the dialing protocol (§4.5) to agree on a set of dead drop nodes in the network, DAENet prohibits adversaries from tracking an honest participant and revealing her identity. We trust the PRNG used in the dialing protocol to generate a series of $DeadDrop_keys$, adversaries cannot predict the next dead drop node to exchange packet payload.

Therefore, defeating DAENet's anti-phishing protocol requires active attackers precisely delay all the on-path application messages in each communication round. Denote the normal averaged end-to-end communication latency as l_1, the delay time as T_d and the expected path length through dead drop as l_2. As the expected communication time through dead drop nodes is fixed, if the phishing attacker receives a delayed message whose latency is $l_1 + (T_d \times l_2)$, then she might have confidence to reveal the message sender.

However, precisely blocking all the on-path message is difficult, meanwhile, the usable time is short. As we will show in §6.1, the expectation of averaged end-to-end latency is less than $2.2s$. To successfully defeat the protocol, the adversary has to precisely predict and delay all the on-path

application messages with probability $1/logN$ for each link within $2.2s$, where N is the total number of participants. Therefore, this attack is beyond the capability of attackers.

Aggresive Hijacking Packets. To de-anonymize network participants, a more aggressive approach is to drop a significant number of packets. For example, active attackers can launch $(n-1)$ attack [57] to track a specific message from Alice by blocking other packets to an honest participant. Also, network adversaries can inject malformed packets to replace ordinary network packets. Note that in this scenario, an honest participant can easily detect such misbehavior and notice a compromised successor in the network. Honest participants can simply rejoin the network to switch to a new location and fetch a new list of neighbors for anonymous messaging.

Under attack or Network Congestion? One possible question in DAENet is how to differentiate between packet dropping due to a compromised participants or network congestion. In theory, both of them can make DAENet lose its liveness while malicious packet drop may also lead to privacy leakage (shown in §4.4). In DAENet, it is not a critical issue to differentiate between these two circumstances because DAENet is not a penalty-based system (e.g., Miranda) that makes compromised participants lose their connections in the network. On the contrary, DAENet detects packets drops to maintain a consistent view of membership in the network, caused by either misbehaves or network congestion, thus honest participants will *not* be wrongly punished.

5.3 Proof of Randomness

Proof of Randomness. DAENet requires a participant to send messages to all her neighbors with the same probability. However, different sending rates can reveal sensitive messaging patterns to GPAs. Next we prove that a DAENet participant sends messages to all her neighbors with the same probability and thus achieves full randomness.

Suppose that each node in an identifier ring $N <= 2^n - 1$ sends message to a random node in the ring. Each node id has $log_2 N$ neighbors, namely $id + 2^i$ for each $i <= n$. Then each neighbor of an arbitrary node id has the same expectation on access time.

Theorem 3. *Each neighbor of an arbitrary node id, denoted as $id + 2^i (0 <= i < n)$, has the same number of access.*

Proof. Suppose that two node x and y are two identical node in the ring, we evaluate one node id, where

$$x \rightarrow ... \rightarrow id \rightarrow ... \rightarrow y. \tag{5}$$

where the identifier of node x may be equal to id.

As the routing from x to y will pass id to y, then we have $x = id - (\sum_{i=0}^{n} X_i 2^i)$, and $y = id + (\sum_{i=0}^{n} Y_i 2^i)$. X_i and Y_i is a selection variable. If a message is routing from x to y, and passes id to its i_0^{th} neighbor of node id, then $X_i = 0, i <= i_0$ and $Y_i = 0, i > i_0$. Therefore, for a neighbour $id + 2^i$ of node id, the total number of (x, y) pair that passes id and $id + 2^i$ is

$$(\sum_{k=0}^{n-1-i} C_{n-1-i}^{k})(\sum_{k=0}^{i} C_i^k) = 2^{n-1-i} 2^i = 2^{n-1} \tag{6}$$

, which is identical to all id's neighbors. □

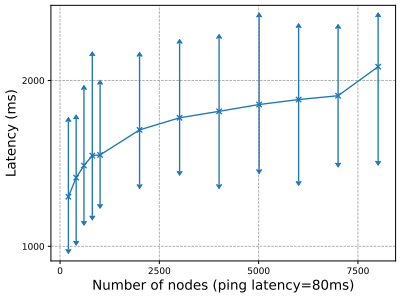


Figure 6: Latency of DAENet when 50 to 8000 participants simultaneously send traffic at rate $\mu = 50ms$ and shuffle messages with probability $\delta = 0.8$. We assume that there is no additional delay add by participants.

5.4 Limited Observable Variables

DAENet's protocol is carefully managed to reveal only a small, yet insensitive set of variables to adversaries. First, by running code inside SGX, we prevent adversaries from directly intervening the protocol execution and seeing the decrypted plaintext of network packets. Malicious participants can monitor traffic links and deduce a set of her predecessors and successors under DAENet's topology. However, she cannot distinguish whether a received packet from a predecessor is a control message - used to maintain the network structure, or an application message - used for communication.

Second, DAENet's shuffling protocol, used for hiding communication circuits, makes all participants run in a stealthy p2p network and exposes just two variables to adversaries: the total number of sent out packets in each round and the output rate of participants. These two variables are insensitive because they cannot reveal which participant is actually talking, as adversaries cannot distinguish an application message. Also, since we unify the output rate of participants, observing the output rate does not reveal any sensitive information as well.

To conclude, DAENet reveals three insensitive observable variables to adversaries: closest neighborhood (i.e., predecessors and successors of a participant), number of sent-out packets in each round, and the output rate. This set is significantly smaller than previous anonymous messaging systems, enabling DAENet to minimize the useful information exposed to adversaries in the network.

6 Evaluation

Our evaluation was conducted on 20 computers with SGX-equipped Intel(R) Xeon(R) CPU E3-1280 v6 with 24 cores, 64GB RAM and 2TB SSD. All computers form a cluster with 40Gbps network. We used `tc` to set intra or inter machines network latency as $40ms$ to simulate Internet network environment. Each machines runs multiple instances of DAENet.

We built a chatting application for evaluating the performance. The chatting clients connect to a server for communications and will send a message to server when it receives the server's reply or a timeout is triggered. The chatting server simply replies to all messages received from clients. We sampled 10% of all participants as clients/servers, and

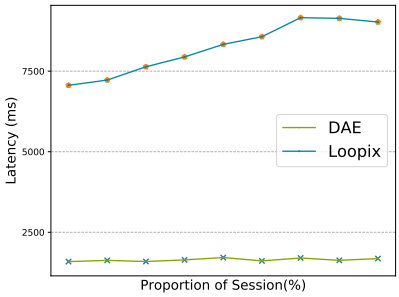


Figure 7: Latency of DAENet for anonymous communicating for varying number of sessions. The latency does *not* increases as the number of session grows.

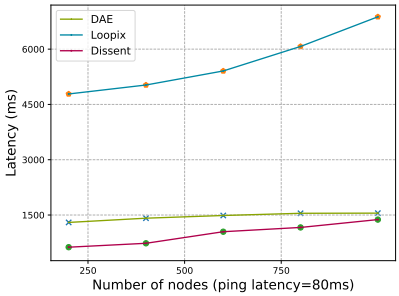


Figure 8: Latency comparison. We measured Loopix and Dissent - two state-of-art scalable anonymous messaging systems.

other participants will work as normal relays. Except for the node scalability evaluation, we use 1,000 participants to evaluate the performance. We set the timeout of message in chatting as 10s.

We compared DAENet with two shuffle-based systems (Loopix and Dissent) which have an approximative security guarantee. Loopix is an open-sourced, shuffle-based anonymous network defending against traffic analysis. We ran 7 mix servers, 10 providers in the clusters and various number of clients in Loopix. We sampled 10% of all Loopix clients to serve as a server or a client of our applications. We wrote an interface to connect the two application (i.e., chatting and file Sharing) to Loopix clients. As Loopix hides the sender's identifier and does not support message reply, we attached the sender id in each message for message reply. We also compared DAENet with Dissent, an open-sourced, shuffle-based anonymous network. We wrote an interface to forward traffic of the Chatting application to Dissent and ran a Dissent private cluster for evaluations.

Quantitatively, our evaluation answers the following research questions:

§6.1 Can DAENet support a large number of participants and provide acceptable performance?
§6.2 How sensitive is DAENet to its parameters?
§6.3 How robust is DAENet to network churn and machine failure?

6.1 Efficiency and Scalability

To analyze the efficiency and scalability of DAENet, we answer the following three research questions in this subsection:

instance/machine	20	40	60	80	100
bandwidth/instance (MB/s)	0.14	0.14	0.14	0.14	0.13

Table 2: Bandwidth cost of running DAENet.

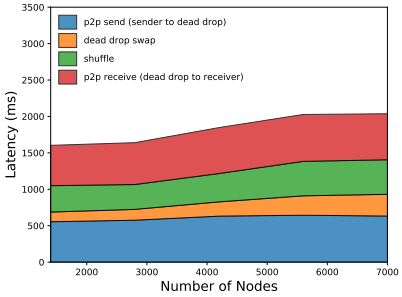


Figure 9: Breakdown of DAENet latency.

- Can DAENet support a large number of users and scale horizontally?
- How does DAENet compare to prior systems?
- Will DAENet slow down the communication?

Horizontally scalability. To demonstrate that DAENet scales horizontally, we measured the end-to-end latency for participants to route million messages as the number of participants varied. As shown in Figure 6, the latency increases logarithmically with increasing number of users. When 8000 participants send traffic simultaneously the latency is nearly 2000ms.

Note that the latency overhead increases logarithmically with total number of participants. This is because the underlying topology of DAENet is a structured p2p network, where the expected path length for one *lookup* request grows logarithmically. In DAENet we utilize Chord, the expected path length for a lookup is $log\mathcal{N}$, where $\mathcal{N}$ is the total number of participants in the network. When DAENet scales to 1M participants, the expected path length for a lookup only grows to 20.

Number of messages. To evaluate how the number of communication session affects latency, we increased the proportion of active nodes (i.e., nodes in communication sessions) from 10% to 95%, and measured the network latency, as shown in Figure 7. As the proportion of active nodes increases, DAENet's latency does not increase much, while Loopix's increases dramatically. This is because Loopix incurred large shuffle overhead while the number of sessions increases, while DAENet can identify them.

Comparison to prior work. To compare DAENet's scalability we ran an experiment in our cluster with 20 servers. To evaluate the support for growing participants, we simulated clients by running multiple ($10 \sim 600$) instances on each machine. For comparison, we also include the latency of Loopix and Dissent as reported in previous subsection which are the only *open-sourced* anonymous messaging system that claims to be scalable to users. We picked the system parameters $\mu = 50ms$ as the message emitting rate of participants in the network, and $\delta = 0.8$ as the shuffle rate to mix real messages and dummy messages.

Figure 8 shows that with 800 users DAENet achieves $1.5X$ higher latency than Dissent, and $5X$ lower latency

compared to Loopix. The reason why DAENet incurs higher latency than Dissent is that Dissent is a centralized system and it statically assigns servers for clients to send their messages, thus clients in Dissent doesn't need to forward messages through several hops and save the time for lookups. However, such design exposes attack surface to DoS all the static servers. DAENet scales better than Loopix because all Loopix traffic must go through a single chain of servers while DAENet requires each participant to only process a fraction of messages in the network.

Latency Breakdown. To investigate DAENet's latency, we break down DAENet's latency incurred by shuffle, dead drop messaging and p2p communications, as shown in Figure 9. Around 69.1% of the latency is from p2p communication, as it requires $log(N)$ steps to locate a node in the network. Dead drop communication contributes 8.4% of the latency. The last source of the latency, message shuffling, incurs only 22.5% of the latency.

As we can see from the breakdown results, DAENet will slow down the communication by adding 30.9% more round-trip latency. However, we believe that DAENet is useful for anonymous online communications, as participants may value stronger privacy guarantee and tolerate the moderate latency.

Bandwidth Usage. We test the bandwidth usage in a cluster of 14 machine where each machine holds several instances running independent DAENet protocol. Table 2 shows the bandwidth usage of participants running DAENet protocol. In this experiment, each test only has one conversation with randomly picked participants from all instances.

To understand the minor bandwidth cost (around 0.14MB/s), DAENet's design crucially avoids heavy usage of network resource for sending dummy messages. This is because we also add p2p control messages to the shuffle pools, so that when participants have to send out a dummy message to a neighbor, she can just replace by sending a control message rather than a useless dummy message.

The cost is independent of the number of participants. With more participants, the number of application messages increases with more conversations in the network, so that the bandwidth cost will not rise rapidly.

6.2 Parameter Sensitivity

To understand how the parameters (i.e., shuffle rate and message emitting rate) affects latency, we varied the minimum shuffle rate and message emitting rate, and measured the latency, as shown in Figure 10 and Table 3. With unified message emitting rate $50ms$, the latency increases dramatically when shuffle rate is decreased. This is because, in each shuffle pool of a neighbor, with a smaller shuffle rate, the probability of popping out a real message to that neighbor becomes smaller and the probability of sending a dummy message to that neighbor becomes larger. That is, a real message will have less chance to be sent out to its destination and the latency increases. Note that with a smaller shuffle rate, DAENet guarantees more obliviousness of output messages, since real messages are fully mixed with dummy messages and a malicious observer is more difficult to distinguish a real message.

When the message emitting rate increases, the latency of messages decreases because a message is popped out of the shuffle pool more quickly with larger emitting rate. However, the descending trend of latency is smoother with large emitting rate. This is because that the latency is also bounded by dead drop swap and p2p communication.

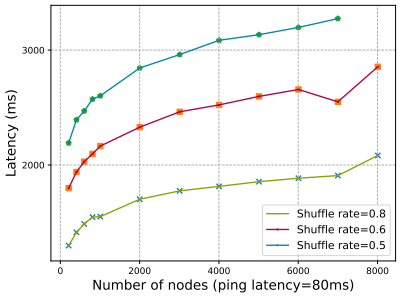


Figure 10: The end-to-end latency of DAENet with unified message emitting rate 50ms and varying shuffle rate.

Message emit rate (μs)	100	500	1000	2000	3000	5000
Latency (ms)	773	758	787	1055	1302	1594

Table 3: The end-to-end latency of DAENet with varying message emitting rate, running in a cluster of 1000 nodes.

6.3 Failure Recovery

Handling node churn is a major issue in p2p systems. To evaluate the failure resilience of DAENet, we ran DAENet for a period of time, with a typical message emitting rate $50ms$ and shuffle rate 0.8, and we arbitrarily killed 10% of all participants for three times (totally killed 30% active participants). To compare the robustness, we also run Loopix to its limit (800 nodes) and arbitrarily kill Loopix nodes as DAENet. The killed nodes of Loopix includes at least one mix nodes to evaluate the failures of centralized servers. The killed nodes of DAENet are sampled uniformly from existing participants, including both active participants (communicating) and idle participants.

Figure 11 and Figure 12 shows the latency before and after killing nodes. When nodes are killed, both DAENet and Loopix's latency becomes extremely high because the lost of transferring message triggers timeout. After that, DAENet's latency resumes to normal in a short time, as DAENet detects failure of message, updates routing table and resumes processing. On the other hand, the performance of Loopix after killing is unstable. This is because client nodes are

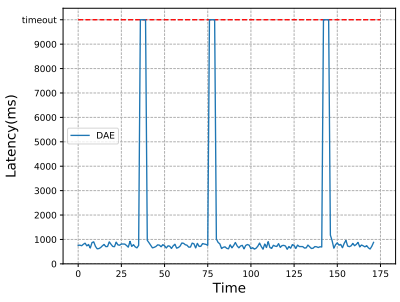


Figure 11: Arbitrarily killing DAENet nodes to simulate network churn with a 10% killing rate.

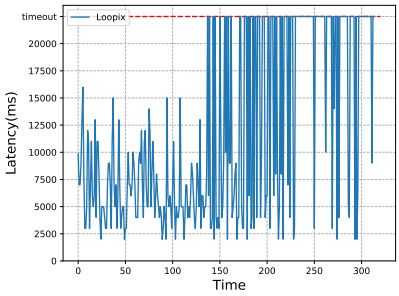


Figure 12: Arbitrarily killing Loopix nodes.

unknown to the failure. In this way, the latency is reduced only when a client node chooses a working server. With more failed server, the Loopix system triggers more timeout and communications become hard to carry on, which may not be tolerated by some users.

7 Related Work

Trusted Execution Environment. TEE provides strong security guarantees (i.e., confidentiality and integrity) for applications with efficiency. Intel SGX [58] is one of the most popular TEE in the market. With the convenience and security properties introduced by SGX, it has been adopted for secure data analysis [53], [59], network analysis [60] and secure key-value stores [61]. SGX-Tor [62] is the first work that applies SGX to anonymous network. Unfortunately, it only defends against specific Tor adversaries and protects circuit/hidden service identifers without protecting users from network-level adversaries.

DC-Net. DC-Net [25], [63], [64], [65] leaks limited sensitive information by using expensive cryptographic primitives. Although it provides stronger security guarantee, it typically incurs large bandwidth consumption and higher latency, making it only suitable for applying to a small scale. To reduce the large latency incurred by DC-Nets, recent work (Pung [22], Atom [29] and Dissent [12]) introduces centralized servers to avoid expensive computation and network bandwidth. However, the centralized servers make DC-Nets vulnerable to DoS attacks. DAENet can defend against both traffic analysis and DoS attacks at the same time, while incurring low latency.

Mix-net Anonymous Network. Mix-nets [11], [66], [67], [68], [69], [70] shuffle network traffic in centralized servers to hide identities of senders and receivers. Although it is easy to implement Mix-nets, most Mix-nets cannot defend statistical traffic analysis, and their formal worse-case guarantees are usually weak [71]. Vuvuzela [47], Stadium [72] and Karaoke [13] adopt differential privacy to defend against traffic analysis. However, all of them cannot defend against active attacks or DoS attacks [73], [74], [75], [76], [77], [78]. Karaoke even stops a whole communication round when one packet is dropped. Another approach is to block all the mis-behaved participants to achieve anonymity, which proves to be practical in some cases [79].

8 Conclusion

DAENet is the first work that enables strong anonymity in a fully decentralized network. DAENet provides both sender-receiver unlinkability under global passive attacks, and sender/receiver anonymity under active attacks. DAENet is built upon a structural p2p network for efficiency and adopts sophisticated techniques, including the dead drop abstraction and shuffling for defending traffic analysis. DAENet makes use of Intel SGX to protect the integrity and confidentiality of the structural network and message shuffles. DAENet's evaluation shows that it scales well with moderate end-to-end latency.

References

[1] A. Johnson, C. Wacek, R. Jansen, M. Sherr, and P. Syverson, "Users get routed: Traffic correlation on tor by realistic adversaries," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. ACM, 2013, pp. 337–348.

[2] B. Resch and J. Engdegård, "Metadata time marking information for indicating a section of an audio object," Aug. 11 2015, uS Patent 9,105,300.

[3] B. Harris and R. Hunt, "Tcp/ip security threats and attack methods," *Computer communications*, vol. 22, no. 10, pp. 885–897, 1999.

[4] R. Jansen, M. Traudt, and N. Hopper, "Privacy-preserving dynamic learning of tor network traffic," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2018, pp. 1944–1961.

[5] R. W. Lai, K.-F. Cheung, S. S. Chow, and A. M. So, "Another look at anonymous communication," *IEEE Transactions on Dependable and Secure Computing*, 2018.

[6] E. Stoycheff, "Under surveillance: Examining facebooks spiral of silence effects in the wake of nsa internet monitoring," *Journalism & Mass Communication Quarterly*, vol. 93, no. 2, pp. 296–311, 2016.

[7] P. Winter and S. Lindskog, *How the great firewall of china is blocking tor*. USENIX-The Advanced Computing Systems Association, 2012.

[8] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," Naval Research Lab Washington DC, Tech. Rep., 2004.

[9] P. Mittal and N. Borisov, "Shadowwalker: peer-to-peer anonymous communication using redundant structured topologies," in *Proceedings of the 16th ACM conference on Computer and communications security*. ACM, 2009, pp. 161–172.

[10] S. J. Murdoch and G. Danezis, "Low-cost traffic analysis of tor," in *2005 IEEE Symposium on Security and Privacy (S&P'05)*. IEEE, 2005, pp. 183–195.

[11] A. M. Piotrowska, J. Hayes, T. Elahi, S. Meiser, and G. Danezis, "The loopix anonymity system," in *26th {USENIX} Security Symposium ({USENIX} Security 17)*, 2017, pp. 1199–1216.

[12] D. I. Wolinsky, H. Corrigan-Gibbs, B. Ford, and A. Johnson, "Dissent in numbers: Making strong anonymity scale," in *Presented as part of the 10th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 12)*, 2012, pp. 179–182.

[13] D. Lazar, Y. Gilad, and N. Zeldovich, "Karaoke: Distributed private messaging immune to passive traffic analysis," in *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*, 2018, pp. 711–725.

[14] H. Corrigan-Gibbs, D. Boneh, and D. Mazières, "Riposte: An anonymous messaging system handling millions of users," in *2015 IEEE Symposium on Security and Privacy*. IEEE, 2015, pp. 321–338.

[15] H. Leibowitz, A. M. Piotrowska, G. Danezis, and A. Herzberg, "No right to remain silent: isolating malicious mixes," in *28th {USENIX} Security Symposium ({USENIX} Security 19)*, 2019, pp. 1841–1858.

[16] D. Lazar, Y. Gilad, and N. Zeldovich, "Yodel: strong metadata security for voice calls," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. ACM, 2019, pp. 211–224.

[17] ——, "Karaoke: Distributed private messaging immune to passive traffic analysis," in *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*, 2018, pp. 711–725.

[18] U. Parampalli, K. Ramchen, and V. Teague, "Efficiently shuffling in public," in *International Workshop on Public Key Cryptography*. Springer, 2012, pp. 431–448.

[19] J. Van Den Hooff, D. Lazar, M. Zaharia, and N. Zeldovich, "Vuvuzela: Scalable private messaging resistant to traffic analysis," in *Proceedings of the 25th Symposium on Operating Systems Principles*. ACM, 2015, pp. 137–152.

[20] L. M. Sumitra and S. C. Miller, "Pathologic gambling disorder: How to help patients curb risky behavior when the future is at stake," *Postgraduate medicine*, vol. 118, no. 1, pp. 31–37, 2005.

[21] P. Paillier *et al.*, "Public-key cryptosystems based on composite degree residuosity classes," in *Eurocrypt*, vol. 99. Springer, 1999, pp. 223–238.

[22] S. Angel and S. Setty, "Unobservable communication over fully untrusted infrastructure," in *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*, 2016, pp. 551–569.

[23] T. Swart and H. Ferreira, "Insertion/deletion correcting coding schemes based on convolution coding," *Electronics Letters*, vol. 38, no. 16, pp. 871–873, 2002.

[24] M. Iansiti and K. R. Lakhani, "The truth about blockchain," *Harvard Business Review*, vol. 95, no. 1, pp. 118–127, 2017.

[25] D. Chaum, "The dining cryptographers problem: Unconditional sender and recipient untraceability," *Journal of cryptology*, vol. 1, no. 1, pp. 65–75, 1988.

[26] S. Kim, J. Han, J. Ha, T. Kim, and D. Han, "Sgx-tor: A secure and practical tor anonymity network with sgx enclaves," *IEEE/ACM Transactions on Networking*, vol. 26, no. 5, pp. 2174–2187, 2018.

[27] A. Mislove, G. Oberoi, A. Post, C. Reis, P. Druschel, and D. S. Wallach, "Ap3: Cooperative, decentralized anonymous communication," in *Proceedings of the 11th workshop on ACM SIGOPS European workshop*. ACM, 2004, p. 30.

[28] D. I. Wolinsky, H. Corrigan-Gibbs, B. Ford, and A. Johnson, "Dissent in numbers: Making strong anonymity scale," in *Presented as part of the 10th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 12)*, 2012, pp. 179–182.

[29] A. Kwon, H. Corrigan-Gibbs, S. Devadas, and B. Ford, "Atom: Horizontally scaling strong anonymity," in *Proceedings of the 26th Symposium on Operating Systems Principles*. ACM, 2017, pp. 406–422.

[30] T. B. Tracker, "Wiki," 2018.

[31] X. Cai, X. C. Zhang, B. Joshi, and R. Johnson, "Touching from a distance: Website fingerprinting attacks and defenses," in *Proceedings of the 2012 ACM conference on Computer and communications security*, 2012, pp. 605–616.

[32] J. Hayes and G. Danezis, "k-fingerprinting: A robust scalable website fingerprinting technique," in *25th {USENIX} Security Symposium ({USENIX} Security 16)*, 2016, pp. 1187–1203.

[33] D. Herrmann, R. Wendolsky, and H. Federrath, "Website fingerprinting: attacking popular privacy enhancing technologies with the multinomial naïve-bayes classifier," in *Proceedings of the 2009 ACM workshop on Cloud computing security*, 2009, pp. 31–42.

[34] S. Li, H. Guo, and N. Hopper, "Measuring information leakage in website fingerprinting attacks and defenses," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1977–1992.

[35] A. Johnson, C. Wacek, R. Jansen, M. Sherr, and P. Syverson, "Users get routed: Traffic correlation on tor by realistic adversaries," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013, pp. 337–348.

[36] Z. Ling, J. Luo, W. Yu, X. Fu, D. Xuan, and W. Jia, "A new cell counter based attack against tor," in *Proceedings of the 16th ACM conference on Computer and communications security*, 2009, pp. 578–589.

[37] Z. Ling, J. Luo, W. Yu, X. Fu, W. Jia, and W. Zhao, "Protocol-level attacks against tor," *Computer Networks*, vol. 57, no. 4, pp. 869–886, 2013.

[38] S. J. Murdoch and P. Zieliński, "Sampled traffic analysis by internet-exchange-level adversaries," in *International workshop on privacy enhancing technologies*. Springer, 2007, pp. 167–183.

[39] N. S. Evans, R. Dingledine, and C. Grothoff, "A practical congestion attack on tor using long paths." in *USENIX Security Symposium*, 2009, pp. 33–50.

[40] J. Geddes, R. Jansen, and N. Hopper, "How low can you go: Balancing performance with anonymity in tor," in *International Symposium on Privacy Enhancing Technologies Symposium*. Springer, 2013, pp. 164–184.

[41] R. Jansen, T. Vaidya, and M. Sherr, "Point break: A study of bandwidth denial-of-service attacks against tor," in *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, Aug. 2019, pp. 1823–1840. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/jansen

[42] R. Jansen, F. Tschorsch, A. Johnson, and B. Scheuermann, "The sniper attack: Anonymously deanonymizing and disabling the tor network," Office of Naval Research Arlington VA, Tech. Rep., 2014.

[43] N. Borisov, G. Danezis, P. Mittal, and P. Tabriz, "Denial of service or denial of security?" in *Proceedings of the 14th ACM conference on Computer and communications security*, 2007, pp. 92–102.

[44] R. Snader and N. Borisov, "A tune-up for tor: Improving security and performance in the tor network." in *ndss*, vol. 8, 2008, p. 127.

[45] A. Panchenko, F. Lanze, and T. Engel, "Improving performance and anonymity in the tor network," in *2012 IEEE 31st International Performance Computing and Communications Conference (IPCCC)*. IEEE, 2012, pp. 1–10.

[46] V. Costan and S. Devadas, "Intel sgx explained." *IACR Cryptology ePrint Archive*, vol. 2016, p. 86, 2016.

[47] J. Van Den Hooff, D. Lazar, M. Zaharia, and N. Zeldovich, "Vuvuzela: Scalable private messaging resistant to traffic analysis," in *Proceedings of the 25th Symposium on Operating Systems Principles*. ACM, 2015, pp. 137–152.

[48] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup service for internet applications," *ACM SIGCOMM Computer Communication Review*, vol. 31, no. 4, pp. 149–160, 2001.

[49] G. Noubir and A. Sanatinia, "Trusted code execution on untrusted platforms using intel sgx," *Virus bulletin*, 2016.

[50] M. Schwarz, S. Weiser, D. Gruss, C. Maurice, and S. Mangard, "Malware guard extension: Using sgx to conceal cache attacks," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2017, pp. 3–24.

[51] M. Orenbach, P. Lifshits, M. Minkin, and M. Silberstein, "Eleos: Exitless os services for sgx enclaves," in *Proceedings of the Twelfth European Conference on Computer Systems*. ACM, 2017, pp. 238–253.

[52] I. Anati, S. Gueron, S. Johnson, and V. Scarlata, "Innovative technology for cpu based attestation and sealing," in *Proceedings of the 2nd international workshop on hardware and architectural support for security and privacy*, vol. 13. ACM New York, NY, USA, 2013.

[53] F. Schuster, M. Costa, C. Fournet, C. Gkantsidis, M. Peinado, G. Mainar-Ruiz, and M. Russinovich, "Vc3: Trustworthy data analytics in the cloud using sgx," in *Security and Privacy (SP), 2015 IEEE Symposium on*. IEEE, 2015, pp. 38–54.

[54] H. Ballani, P. Francis, and X. Zhang, "A study of prefix hijacking and interception in the internet," *ACM SIGCOMM Computer Communication Review*, vol. 37, no. 4, pp. 265–276, 2007.

[55] Y. Gilad and A. Herzberg, "Spying in the dark: Tcp and tor traffic analysis," in *International symposium on privacy enhancing technologies symposium*. Springer, 2012, pp. 100–119.

[56] M. Matsumoto and T. Nishimura, "Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator," *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, vol. 8, no. 1, pp. 3–30, 1998.

[57] A. Serjantov, R. Dingledine, and P. Syverson, "From a trickle to a flood: Active attacks on several mix types," in *International Workshop on Information Hiding*. Springer, 2002, pp. 36–52.

[58] Intel, "Software guard extensions programming reference," https://software.intel.com/sites/default/files/329298-001.pdf.

[59] W. Zheng, A. Dave, J. G. Beekman, R. A. Popa, J. E. Gonzalez, and I. Stoica, "Opaque: An oblivious and encrypted distributed analytics platform." in *NSDI*, 2017, pp. 283–298.

[60] M.-W. Shih, M. Kumar, T. Kim, and A. Gavrilovska, "S-nfv: Securing nfv states by using sgx," in *Proceedings of the 2016 ACM International Workshop on Security in Software Defined Networks & Network Function Virtualization*. ACM, 2016, pp. 45–48.

[61] C. Priebe, K. Vaswani, and M. Costa, "Enclavedb: A secure database using sgx," in *EnclaveDB: A Secure Database using SGX*. IEEE, 2018, p. 0.

[62] S. M. Kim, J. Han, J. Ha, T. Kim, and D. Han, "Enhancing security and privacy of tor's ecosystem by using trusted execution environments." in *NSDI*, 2017, pp. 145–161.

[63] D. Mills, "Dcnet internet clock service," Tech. Rep., 1981.

[64] M. F. Iacchetti, G. D. Marques, and R. Perini, "Operation and design issues of a doubly fed induction generator stator connected to a dc net by a diode rectifier," *IET Electric power applications*, vol. 8, no. 8, pp. 310–319, 2014.

[65] M. Ranjram and P. W. Lehn, "A multiport power-flow controller for dc transmission grids," *IEEE Transactions on Power Delivery*, vol. 31, no. 1, pp. 389–396, 2015.

[66] P. Golle and A. Juels, "Parallel mixing," in *Proceedings of the 11th ACM conference on Computer and communications security*. ACM, 2004, pp. 220–226.
[67] G. Danezis, R. Dingledine, and N. Mathewson, "Mixminion: Design of a type iii anonymous remailer protocol," in *2003 Symposium on Security and Privacy, 2003.* IEEE, 2003, pp. 2–15.
[68] S. Davies and A. Moore, "Mix-nets: Factored mixtures of gaussians in bayesian networks with mixed continuous and discrete variables," in *Proceedings of the Sixteenth conference on Uncertainty in artificial intelligence*. Morgan Kaufmann Publishers Inc., 2000, pp. 168–175.
[69] R. Küsters, T. Truderung, and A. Vogt, "Formal analysis of chaumian mix nets with randomized partial checking," in *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014, pp. 343–358.
[70] M.-C. Silaghi, "Zero-knowledge proofs for mix-nets of secret shares and a version of elgamal with modular homomorphism." *IACR Cryptology ePrint Archive*, vol. 2005, p. 79, 2005.
[71] C. Diaz and A. Serjantov, "Generalising mixes," in *International Workshop on Privacy Enhancing Technologies*. Springer, 2003, pp. 18–31.
[72] N. Tyagi, Y. Gilad, D. Leung, M. Zaharia, and N. Zeldovich, "Stadium: A distributed metadata-private messaging system," in *Proceedings of the 26th Symposium on Operating Systems Principles*. ACM, 2017, pp. 423–440.
[73] K. Park and H. Lee, "On the effectiveness of route-based packet filtering for distributed dos attack prevention in power-law internets," in *ACM SIGCOMM computer communication review*, vol. 31, no. 4. ACM, 2001, pp. 15–26.
[74] H. Zhang, P. Cheng, L. Shi, and J. Chen, "Optimal dos attack scheduling in wireless networked control system," *IEEE Transactions on Control Systems Technology*, vol. 24, no. 3, pp. 843–852, 2015.
[75] H. Wang, L. Xu, and G. Gu, "Floodguard: A dos attack prevention extension in software-defined networks," in *2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. IEEE, 2015, pp. 239–250.
[76] J. E. Belissent, "Method and apparatus for preventing a denial of service (dos) attack by selectively throttling tcp/ip requests," Sep. 7 2004, uS Patent 6,789,203.
[77] Y. Zhang, Z. M. Mao, and J. Wang, "Low-rate tcp-targeted dos attack disrupts internet routing." in *NDSS*. Citeseer, 2007.
[78] H. Liu, "A new form of dos attack in a cloud and its avoidance mechanism," in *Proceedings of the 2010 ACM workshop on Cloud computing security workshop*. ACM, 2010, pp. 65–76.
[79] P. P. Tsang, A. Kapadia, C. Cornelius, and S. W. Smith, "Nymble: Blocking misbehaving users in anonymizing networks," *IEEE Transactions on Dependable and Secure Computing*, vol. 8, no. 2, pp. 256–269, 2009.



Tianxiang Shen is a PhD student supervised by Dr. Heming Cui in Computer Science Department at the University of Hong Kong. His research interests lie in distributed systems, with a particular focus on the system security and data privacy. Prior to his current program, he received B.Eng degree from JiLin University the Excellent Engineering Class.



Jianyu Jiang is currently a third year PhD student in Computer Science Department at The University of Hong Kong. He is working on topics in large scale computation platform under the supervision of Dr. Heming Cui. Prior to his current program, Jianyu receive his Bachelor's Degree in Computer Science Department at Xi'an Jiaotong University, under the supervision of Professor Qi Yong.



Yunpeng Jiang is currently pursuing his Bachelor degree in information security at Department of Computer Science and Engineering, South China University of Technology. His research interests includes system security and trusted computing tecnhiques.



Xusheng Chen received his Bachelor degree in HKU. He is currently a PhD student in Computer Science of HKU (2017-now). He is under the supervision of Dr. Heming Cui. His research interests include distributed consensus protocols, distributed systems and system security.



Ji Qi received his B.S (2015) degree from Beijing Institude of Technology, Beijing, China, and his M.S (2018) degree from Tsinghua University, Beijin, China. He is currently pursuing the PhD in computer science at the University of Hong Kong under the supervision of Dr. Heming Cui. His interests include domain-specific modeling, distributed system and cloud computing.



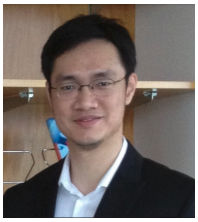
Shixiong Zhao received his Bachelor degree in HKU and his master degree in HKUST. He is currently a PhD student in Computer Science of HKU (2017-now). He is under the supervision of Dr. Heming Cui. His research interests include distributed systems for high performance computing, distributed systems and system security.



Fengwei Zhang is an Associate Professor at Department of Computer Science and Engineering at Southern University of Science and Technology (SUSTech). His primary research interests are in the areas of systems security, with a focus on trustworthy execution, hardware-assisted security, debugging transparency, transportation security, and plausible deniability encryption. Before joining SUSTech, he spent four wonderful years as an Assistant Professor at Department of Computer Science at Wayne State University.



Xiapu Luo received his B.S. in Communication Engineering and M.S. in Communications and Information Systems from Wuhan University. He obtained his Ph.D. degree in Computer Science from the Hong Kong Polytechnic University, under the supervision of Prof. Rocky K.C. Chang. After that, he spent two years at the Georgia Institute of Technology as a post-doctoral research fellow advised by Prof. Wenke Lee. His current research interests include Network and System Security, Information Privacy, Internet Measurement, Cloud Computing, and Mobile Networks.



Heming Cui is an assistant professor in computer science of HKU. His research interests include operating systems, programming languages, distributed systems, and cloud computing, with a particular focus on building software infrastructures and tools to improve reliability and security of real-world software. Homepage: https://i.cs.hku.hk/ heming/. He is a member of IEEE.